AIN SHAMS UNIVERSITY
FACULTY OF ENGINEERING

OS_Memory_Segmentation_Simulator

Operating Systems (CSE335s)

By: Mohamed Khaled Ahmed

Submitted to: Dr/Saher, Eng/Hala, Eng/Habiba

Contents

1 Introduction

The **OS Memory Segmentation Simulator** is a desktop GUI application that simulates how an operating system manages memory using **segmentation**. Segmentation is a memory-management scheme that supports the user's view of a program as a collection of logical segments (Code, Data, Stack, etc.), each with its own size and base address.

1.1 Core Features

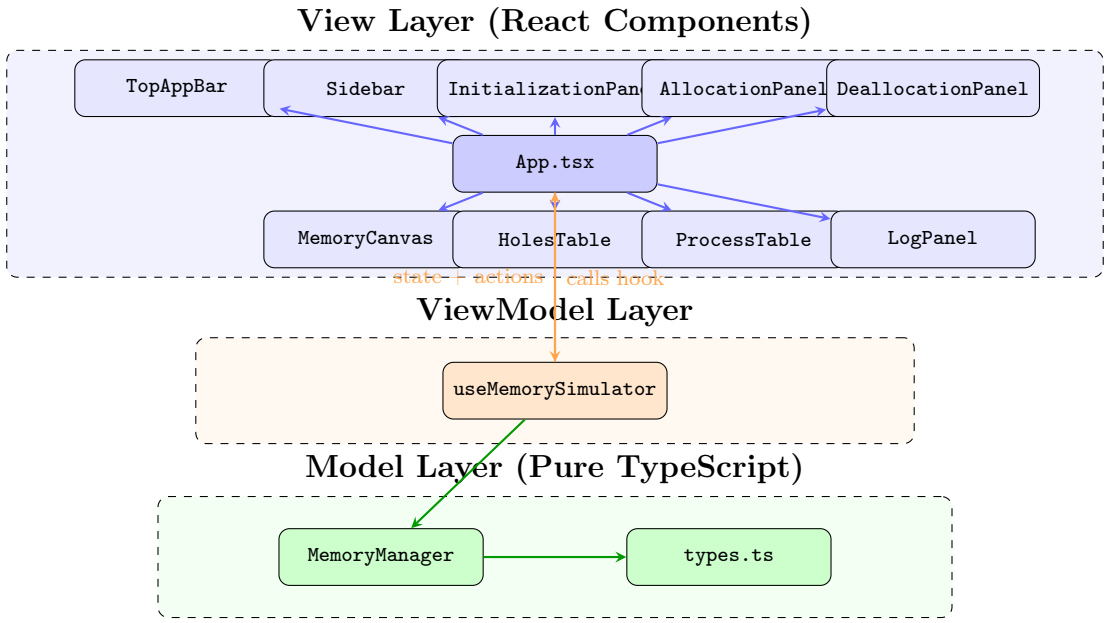
- **Dynamic Memory Initialization** – The user defines the total memory size and the initial set of free partitions (holes) with their starting addresses and sizes.
- **Process Allocation** – Processes are allocated one at a time. Each process has a name and one or more named segments with specific sizes. The user selects the allocation strategy: *First-Fit* or *Best-Fit*.
- **Process Deallocation with Coalescing** – When a process is deallocated, all its segments are returned as free holes. Adjacent holes are automatically merged (coalesced) to combat external fragmentation.
- **Transactional Allocation** – If any segment of a process cannot fit, the entire allocation is rolled back. No partial allocations occur.
- **Live Memory Visualization** – A real-time color-coded memory canvas shows allocated segments, free holes, and system-occupied regions.
- **Segment Tables** – Each allocated process has its own segment table showing the segment name, size, and base address.
- **Free Partitions Table** – A live table of all current holes with their start addresses and sizes.
- **Event Logging** – Every allocation, deallocation, and error is logged with timestamps.
- **Cross-Platform Desktop App** – Built with Electron, runs as a native desktop application on Windows, macOS, and Linux.

2 System Architecture

The project follows a strict **MVVM (Model–View–ViewModel)** architecture pattern to enforce clean separation of concerns:

Layer	Responsibility	Files
Model	Pure OS logic, zero UI knowledge	`src/model/*.ts`
ViewModel	Bridges Model to View via React hooks	`src/viewmodel/*.ts`
View	Renders UI, never calls Model directly	`src/components/*.tsx`

2.1 Architecture Diagram



2.2 Data Flow

Data flows in one direction (unidirectional):

1. The **View** calls an action (e.g., `actions.allocate(...)`).
2. The **ViewModel** hook forwards it to the **Model** (`MemoryManager.allocate(...)`).
3. The Model performs pure logic and returns a result.
4. The ViewModel updates React state via `setState`.
5. React re-renders the View with the new data.

2.3 Folder Structure

```
OS_Memory_Segmentation_Simulator/
  electron/
    main.ts              # Electron main process
    preload.ts           # Secure context bridge
  src/
    model/
      types.ts           # TypeScript interfaces
      MemoryManager.ts   # First-Fit, Best-Fit, Coalescing
    viewmodel/
```

```
    useMemorySimulator.ts  # React hook (ViewModel)
  components/
    TopAppBar.tsx       # Header with system status
    Sidebar.tsx         # Navigation panel
    InitializationPanel.tsx
    AllocationPanel.tsx
    DeallocationPanel.tsx
    MemoryCanvas.tsx    # Visual memory map
    HolesTable.tsx      # Free partitions table
    ProcessTable.tsx    # Per-process segment tables
    LogPanel.tsx        # Event log
  App.tsx               # Root component
  main.tsx              # React entry point
  index.css             # Tailwind styles
```

3 Model Layer (src/model/)

The Model layer contains pure TypeScript logic with zero React dependencies. It implements the core OS memory management algorithms.

3.1 types.ts – Domain Interfaces

This file defines all data structures used throughout the application. These are pure type definitions with no runtime behavior.

3.1.1 Allocation Strategy Type

Defines the two supported algorithms as a union type:

```
export type AllocationStrategy = "first-fit" | "best-fit";
```

3.1.2 Core Data Interfaces

`Hole` represents a free memory partition. `Segment` represents one allocated segment of a process. `Process` groups segments under a process name.

```
export interface Hole {
  start: number;
  size: number;
}

export interface Segment {
  name: string;
  size: number;
  base: number; // starting address once allocated
}

export interface Process {
  name: string;
  segments: Segment[];
}
```

3.1.3 Memory Block (Discriminated Union)

Used for visualization. The `kind` field enables TypeScript to narrow the type at compile time:

```
export type MemoryBlock =
  | { kind: "hole"; start: number; size: number }
  | { kind: "segment"; start: number; size: number;
      processName: string; segmentName: string; }
  | { kind: "occupied"; start: number; size: number };
```

3.1.4 State and Result Types

```
export interface MemoryState {
  totalMemory: number;
  holes: Hole[];
  processes: Process[];
  memoryMap: MemoryBlock[];
}

export interface AllocationResult {
```

```
  success: boolean;
  message: string;
}
```

3.2 `MemoryManager.ts` – Core Logic Engine

This class encapsulates all OS memory management logic. It maintains two canonical tables: the holes table (free partitions) and the processes table (allocated partitions with segment tables).

3.2.1 Class Declaration and Constructor

The constructor takes total memory and initial holes, sorts them by address:

```
export class MemoryManager {
  readonly totalMemory: number;
  private _holes: Hole[];
  private _processes: Process[];

  constructor(totalMemory: number, initialHoles: Hole[]) {
    this.totalMemory = totalMemory;
    this._holes = [...initialHoles].sort((a, b) => a.start - b.start);
    this._processes = [];
  }
```

3.2.2 Read-Only Accessors

Return deep copies to prevent external mutation:

```
  get holes(): Hole[] {
    return this._holes.map((h) => ({ ...h }));
  }

  get processes(): Process[] {
    return this._processes.map((p) => ({
      name: p.name,
      segments: p.segments.map((s) => ({ ...s })),
    }));
  }
```

3.2.3 Allocation Algorithm

Works on a copy of holes (transactional rollback on failure). For each segment, it finds a suitable hole using the chosen strategy, places the segment, and shrinks or removes the hole:

```
  allocate(
    processName: string,
    segmentDefs: { name: string; size: number }[],
    strategy: AllocationStrategy
  ): AllocationResult {
    if (this._processes.some((p) => p.name === processName)) {
      return { success: false,
        message: `Process "${processName}" already exists.` };
    }

    let workingHoles = this._holes.map((h) => ({ ...h }));
    const allocatedSegments: Segment[] = [];

    for (const seg of segmentDefs) {
      const holeIndex = this.findHole(workingHoles, seg.size, strategy);
      if (holeIndex === -1) {
        return { success: false,
          message: `Process "${processName}" does not fit. ` +
          `Segment "${seg.name}" (size ${seg.size}) cannot be
           placed in any available hole.` };
      }
      const hole = workingHoles[holeIndex];
```

```
      allocatedSegments.push(
        { name: seg.name, size: seg.size, base: hole.start });

      if (hole.size === seg.size) {
        workingHoles.splice(holeIndex, 1);
      } else {
        workingHoles[holeIndex] = {
          start: hole.start + seg.size,
          size: hole.size - seg.size,
        };
      }
    }
    this._holes = workingHoles;
    this._processes.push(
      { name: processName, segments: allocatedSegments });
    return { success: true,
      message: `Process "${processName}" allocated successfully.` };
  }
```

3.2.4 Deallocation with Coalescing

Returns all segments as holes, sorts, then merges adjacent ones:

```
  deallocate(processName: string): AllocationResult {
    const idx = this._processes.findIndex(
      (p) => p.name === processName);
    if (idx === -1) {
      return { success: false,
        message: `Process "${processName}" not found.` };
    }
    const process = this._processes[idx];
    for (const seg of process.segments) {
      this._holes.push({ start: seg.base, size: seg.size });
    }
    this._processes.splice(idx, 1);
    this._holes.sort((a, b) => a.start - b.start);
    this.coalesce();
    return { success: true,
      message: `Process "${processName}" deallocated.` };
  }
```

3.2.5 First-Fit and Best-Fit Strategy

First-Fit returns the first hole large enough. **Best-Fit** scans all holes and returns the smallest sufficient one:

```
  private findHole(holes: Hole[], size: number,
    strategy: AllocationStrategy): number {
    if (strategy === "first-fit") {
      return holes.findIndex((h) => h.size >= size);
    }
    let bestIdx = -1;
    let bestSize = Infinity;
    for (let i = 0; i < holes.length; i++) {
      if (holes[i].size >= size && holes[i].size < bestSize) {
        bestSize = holes[i].size;
        bestIdx = i;
      }
    }
    return bestIdx;
  }
```

3.2.6 Coalescing Adjacent Holes

Merges two holes when `hole[i].start + hole[i].size === hole[i+1].start`:

```
  private coalesce(): void {
    if (this._holes.length <= 1) return;
    const merged: Hole[] = [this._holes[0]];
    for (let i = 1; i < this._holes.length; i++) {
      const prev = merged[merged.length - 1];
      const curr = this._holes[i];
      if (prev.start + prev.size === curr.start) {
        prev.size += curr.size;
      } else {
        merged.push(curr);
      }
    }
    this._holes = merged;
  }
```

3.2.7 Memory Map Builder

Constructs a sorted array of all memory blocks for visualization, filling gaps with "occupied" blocks:

```
  buildMemoryMap(): MemoryBlock[] {
    const blocks: MemoryBlock[] = [];
    for (const h of this._holes) {
      blocks.push({ kind: "hole", start: h.start, size: h.size });
    }
    for (const p of this._processes) {
      for (const s of p.segments) {
        blocks.push({ kind: "segment", start: s.base,
          size: s.size, processName: p.name,
          segmentName: s.name });
      }
    }
    blocks.sort((a, b) => a.start - b.start);
    const filled: MemoryBlock[] = [];
    let cursor = 0;
    for (const block of blocks) {
      if (block.start > cursor) {
        filled.push({ kind: "occupied", start: cursor,
          size: block.start - cursor });
      }
      filled.push(block);
      cursor = block.start + block.size;
    }
    if (cursor < this.totalMemory) {
      filled.push({ kind: "occupied", start: cursor,
        size: this.totalMemory - cursor });
    }
    return filled;
  }
}
```

4 ViewModel Layer (src/viewmodel/)

The ViewModel bridges the pure Model logic to the React View using custom hooks.

4.1 useMemorySimulator.ts – The ViewModel Hook

This custom hook owns a MemoryManager instance via useRef and exposes reactive state plus action functions to the View.

4.1.1 Exposed Types

Defines the state shape and action signatures that the View consumes:

```
export interface LogEntry {
  id: number;
  kind: "success" | "error" | "info";
  text: string;
}

export interface SimulatorState {
  initialized: boolean;
  totalMemory: number;
  holes: Hole[];
  processes: Process[];
  memoryMap: MemoryBlock[];
  logs: LogEntry[];
}

export interface SimulatorActions {
  initialize: (totalMemory: number, holes: Hole[]) => void;
  allocate: (processName: string,
    segments: { name: string; size: number }[],
    strategy: AllocationStrategy) => void;
  deallocate: (processName: string) => void;
  reset: () => void;
}
```

4.1.2 Hook Function and State Management

useState creates reactive state. useRef stores the mutable MemoryManager without triggering re-renders:

```
export function useMemorySimulator():
  [SimulatorState, SimulatorActions] {
  const [state, setState] = useState<SimulatorState>(EMPTY_STATE);
  const managerRef = useRef<MemoryManager | null>(null);
  const logIdRef = useRef(0);
```

4.1.3 Logging Utility

Prepends new log entries and caps at 50:

```
  const addLog = useCallback(
    (kind: LogEntry["kind"], text: string) => {
      const entry: LogEntry =
        { id: ++logIdRef.current, kind, text };
      setState((prev) => ({ ...prev,
        logs: [entry, ...prev.logs].slice(0, 50) }));
    }, []);
```

4.1.4 Initialize Action

Creates a new `MemoryManager` and snapshots its state into React:

```
  const initialize = useCallback(
    (totalMemory: number, holes: Hole[]) => {
      const mgr = new MemoryManager(totalMemory, holes);
      managerRef.current = mgr;
      setState({
        initialized: true, totalMemory,
        holes: mgr.holes, processes: [],
        memoryMap: mgr.buildMemoryMap(), logs: [],
      });
      logIdRef.current = 0;
      addLog("info",
        `Memory initialized: ${totalMemory}K total,
         ${holes.length} hole(s).`);
    }, [addLog]);
```

4.1.5 Allocate and Deallocate Actions

Both follow the same pattern: call the Model, snapshot state, log result:

```
  const allocate = useCallback(
    (processName: string,
     segments: { name: string; size: number }[],
     strategy: AllocationStrategy) => {
      const mgr = managerRef.current;
      if (!mgr) return;
      const result = mgr.allocate(
        processName, segments, strategy);
      setState((prev) => ({ ...prev, ...snapshot() }));
      addLog(result.success ? "success" : "error",
        result.message);
    }, [addLog, snapshot]);

  const deallocate = useCallback(
    (processName: string) => {
      const mgr = managerRef.current;
      if (!mgr) return;
      const result = mgr.deallocate(processName);
      setState((prev) => ({ ...prev, ...snapshot() }));
      addLog(result.success ? "success" : "error",
        result.message);
    }, [addLog, snapshot]);

  return [state, { initialize, allocate, deallocate, reset }];
}
```

5 View Layer (src/components/)

The View layer consists of modular React components. Each component receives data and callbacks through **props** (unidirectional data flow).

5.1 App.tsx – Root Orchestrator

The root component calls the `useMemorySimulator` hook and distributes state and actions to all child components. It manages which control panel (Initialization, Allocation, or Deallocation) is active via a sidebar selection state. It also maintains a running list of all process names ever seen for stable color assignment.

5.2 TopAppBar.tsx – Header Bar

Displays the application title "MEM_Manager" and a system status indicator that shows "AWAITING_INIT" before initialization and "SYSTEM_STABLE" after.

5.3 Sidebar.tsx – Navigation Panel

A vertical navigation panel with three buttons: Initialization, Allocation, and Deallocation. The active panel is highlighted with a green accent border. Clicking a button switches the left pane content.

5.4 InitializationPanel.tsx – System Setup Form

Provides inputs for total memory size and initial holes (start address + size). Holes can be added and removed before initialization. Once the "Initialize Memory" button is clicked, the form locks and the system begins accepting allocations.

5.5 AllocationPanel.tsx – Process Allocation Form

Contains inputs for process name, number of segments, and a dynamic grid of segment name/size inputs. The number of segment rows adjusts automatically. The user selects First-Fit or Best-Fit from a dropdown, then clicks "Allocate Process."

5.6 DeallocationPanel.tsx – Process Deallocation

A dropdown of all currently allocated processes. Selecting one and clicking "Deallocate" frees all its segments and triggers coalescing.

5.7 MemoryCanvas.tsx – Visual Memory Map

The most visually complex component. Renders each `MemoryBlock` as a colored div whose height is proportional to its size via flexbox. Segments are color-coded per process, holes show a crosshatch pattern, and occupied regions appear dimmed. Tooltips on hover show start address and size.

5.8 `HolesTable.tsx` – Free Partitions Table

A data table displaying all current holes with ID, start address, and size columns.

5.9 `ProcessTable.tsx` – Segment Tables

Renders one color-coded table per allocated process, showing each segment's name, size, and base address.

5.10 `LogPanel.tsx` – Event Log

Displays a scrollable log of events (success in green, errors in red, info in blue) with the most recent entry at the top.

6 Testing

The following test cases were run as specified in the assignment. Both use the same initial setup:

- **Total Memory:** 1000K
- **Initial Holes:** H1 (Start=0, Size=300), H2 (Start=400, Size=250), H3 (Start=700, Size=200)

Operations performed in order:

1. Allocate P1: Code=100, Data=120, Stack=90
2. Allocate P2: Code=200, Data=40
3. Allocate P3: Code=120, Data=50
4. Deallocate P1
5. Allocate P4: Code=230, Data=40

6.1 Test Case 1: First-Fit Algorithm

6.1.1 Step 1 – Initialize Memory with Holes

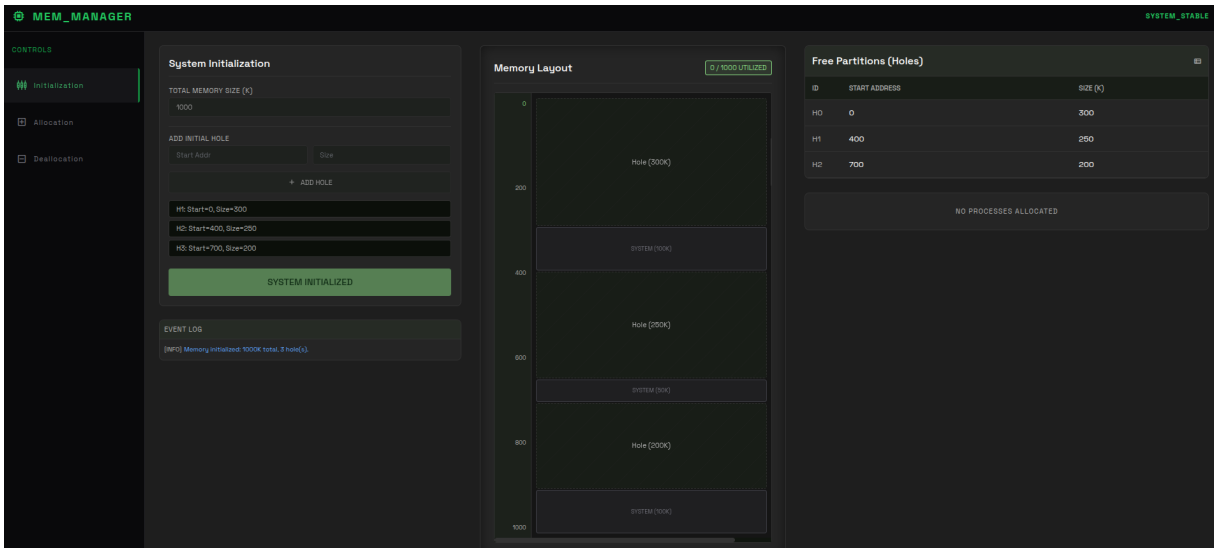


Figure 1: Memory initialized with 1000K total and 3 holes.

6.1.2 Step 2 – Allocate Process P1 (Code=100, Data=120, Stack=90)

P1's segments are placed using First-Fit: Code at 0 (in H1), Data at 100 (in H1), Stack at 400 (in H2, since H1's remainder is only 80K).

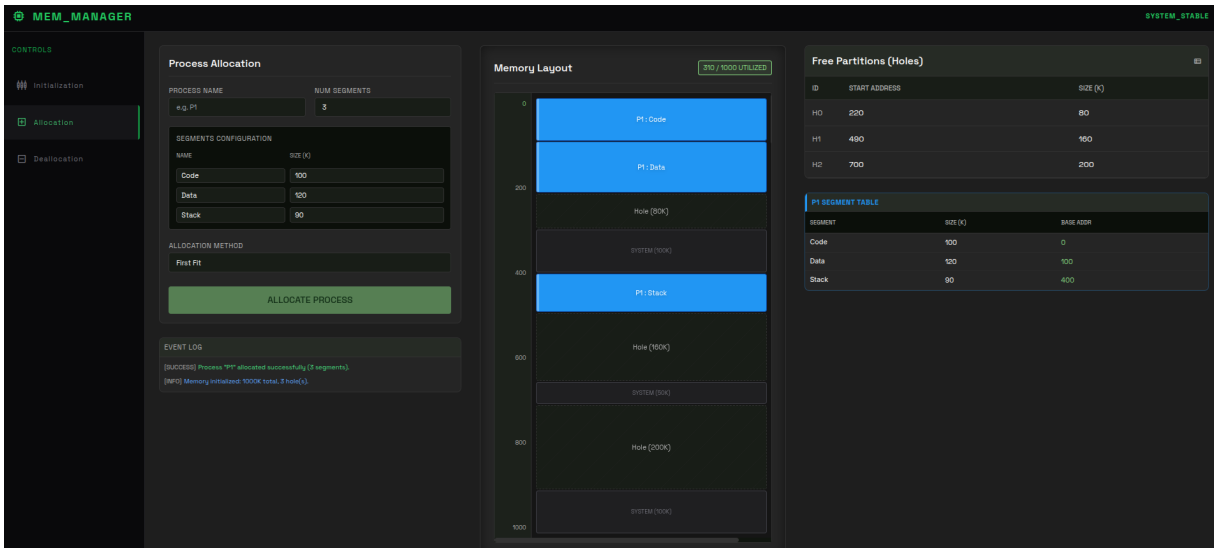


Figure 2: P1 allocated with First-Fit.

6.1.3 Step 3 – Allocate Process P2 (Code=200, Data=40)

Code=200 scans H1(80)→skip, H2(160)→skip, H3(200)→exact fit. Data=40 goes to H1(80).

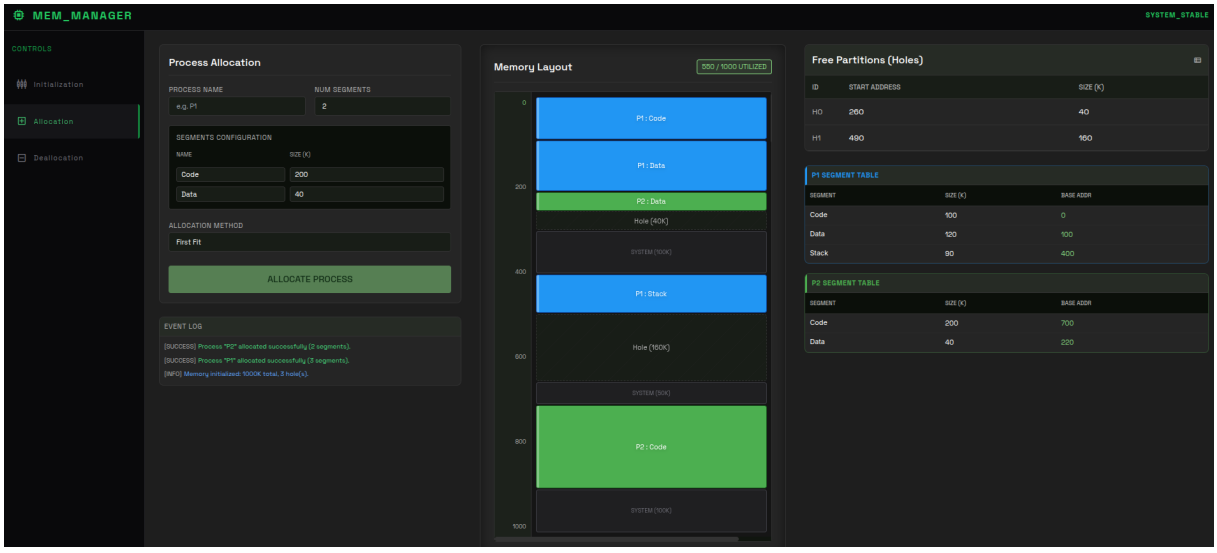


Figure 3: P2 allocated with First-Fit.

6.1.4 Step 4 – Allocate Process P3 (Code=120, Data=50) – FAILS

Code=120 fits in H2(160), but then Data=50 cannot fit in any remaining hole (H1=40, H2=40). Transaction rolled back.

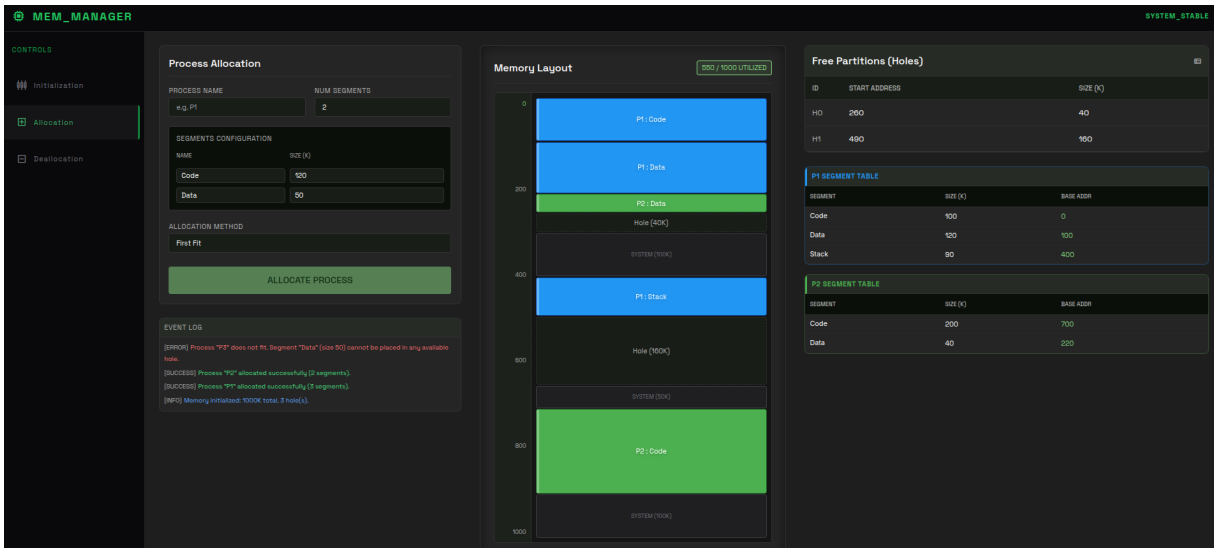


Figure 4: P3 allocation fails – insufficient contiguous memory.

6.1.5 Step 5 – Deallocate Process P1

P1's 3 segments become holes. Adjacent holes coalesce: (0,100)+(100,120)→(0,220) and (400,90)+(490,160)→(400,250).

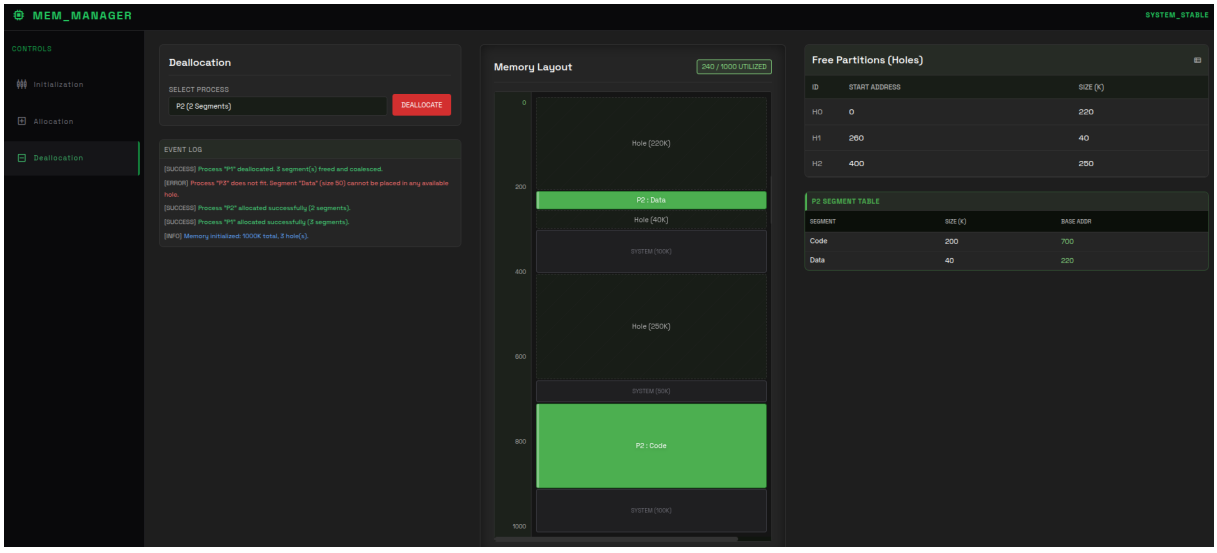


Figure 5: P1 deallocated with coalescing.

6.1.6 Step 6 – Allocate Process P4 (Code=230, Data=40)

Code=230 fits in the coalesced hole at (400,250). Data=40 goes to (0,220). P4 succeeds.

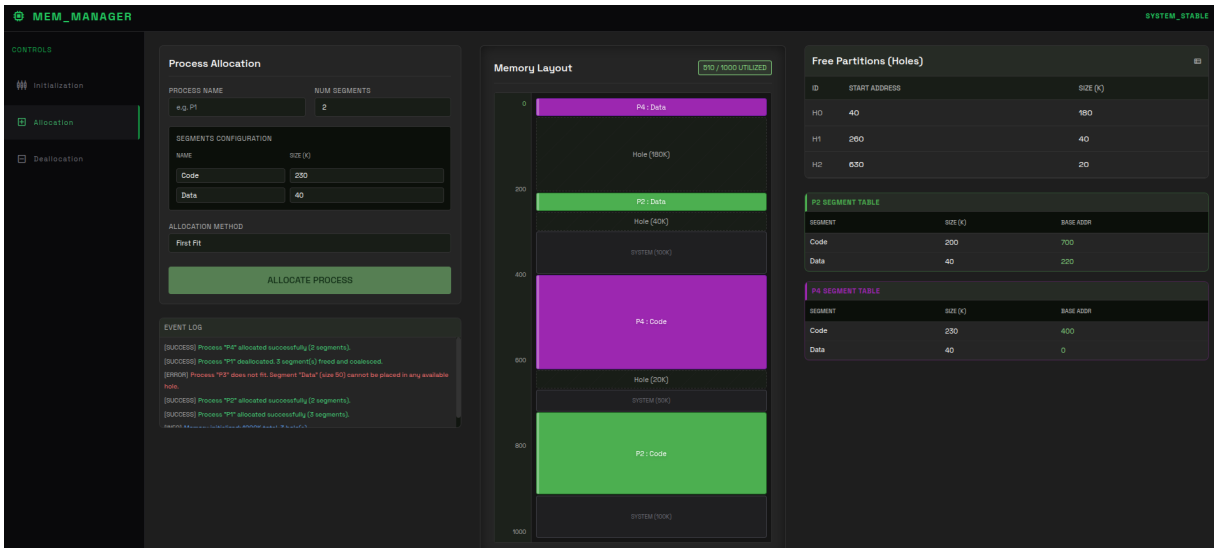


Figure 6: P4 allocated with First-Fit after P1 deallocation.

6.2 Test Case 2: Best-Fit Algorithm

6.2.1 Step 1 – Allocate Process P1 (Code=100, Data=120, Stack=90)

Best-Fit picks the smallest sufficient hole: Code→H3(200), Data→H2(250), Stack→H3's remainder(100).

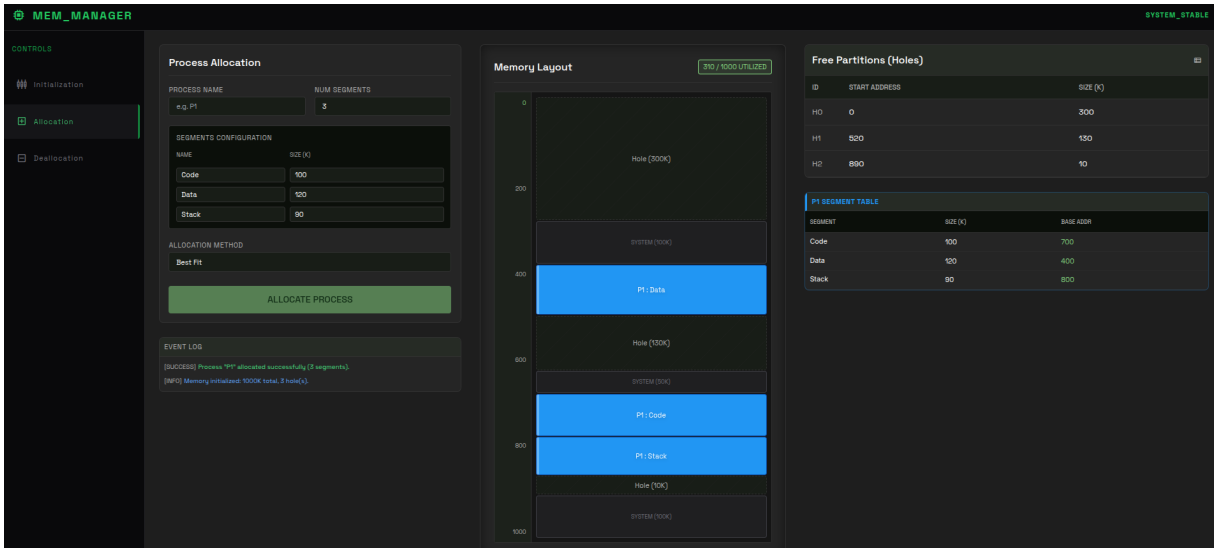


Figure 7: P1 allocated with Best-Fit.

6.2.2 Step 2 – Allocate Process P2 (Code=200, Data=40)

Code=200 best fits in H1(300). Data=40 best fits in H1's remainder(100).

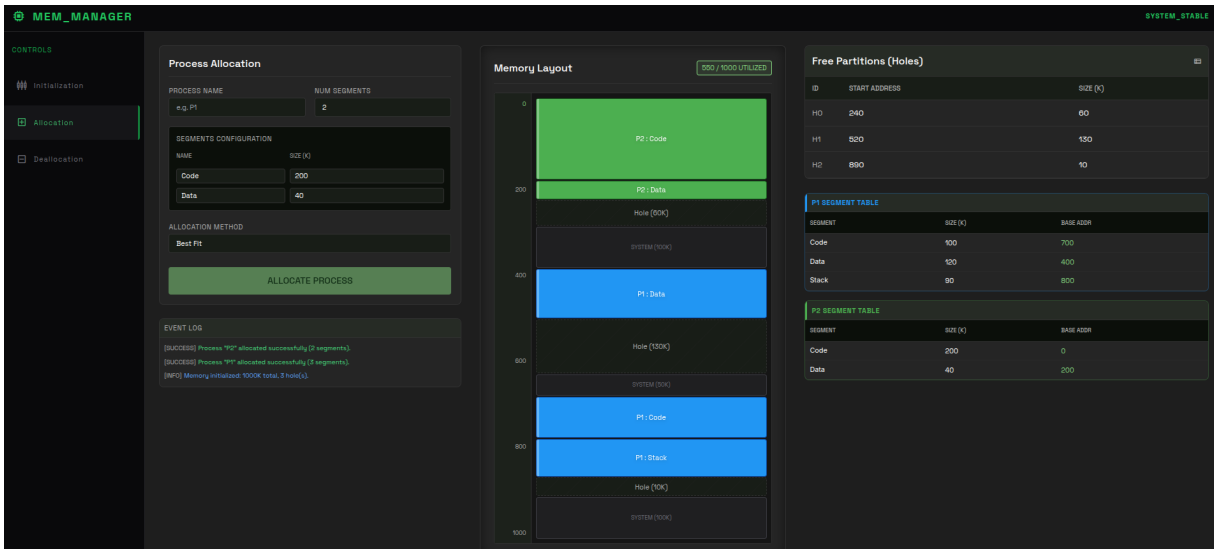


Figure 8: P2 allocated with Best-Fit.

6.2.3 Step 3 – Allocate Process P3 (Code=120, Data=50)

Code=120 fits in (520,130). Data=50 fits in (240,60). **P3 succeeds with Best-Fit** (it failed with First-Fit).

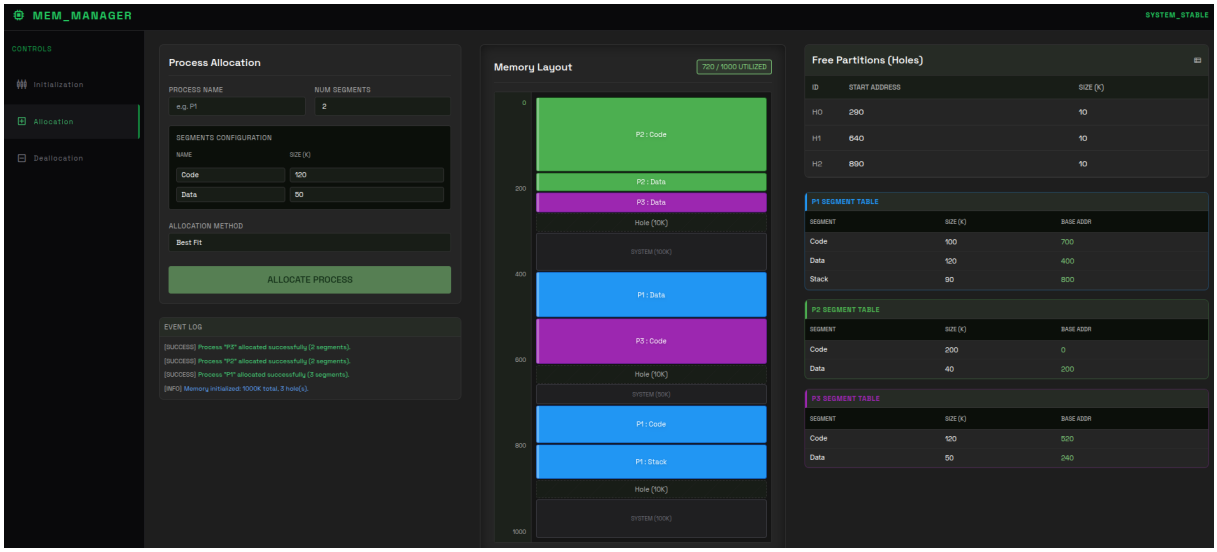


Figure 9: P3 allocated with Best-Fit – succeeds where First-Fit failed.

6.2.4 Step 4 – Deallocate Process P1

P1's segments freed. Adjacent holes at (700,100)+(800,90)+(890,10) coalesce to (700,200).

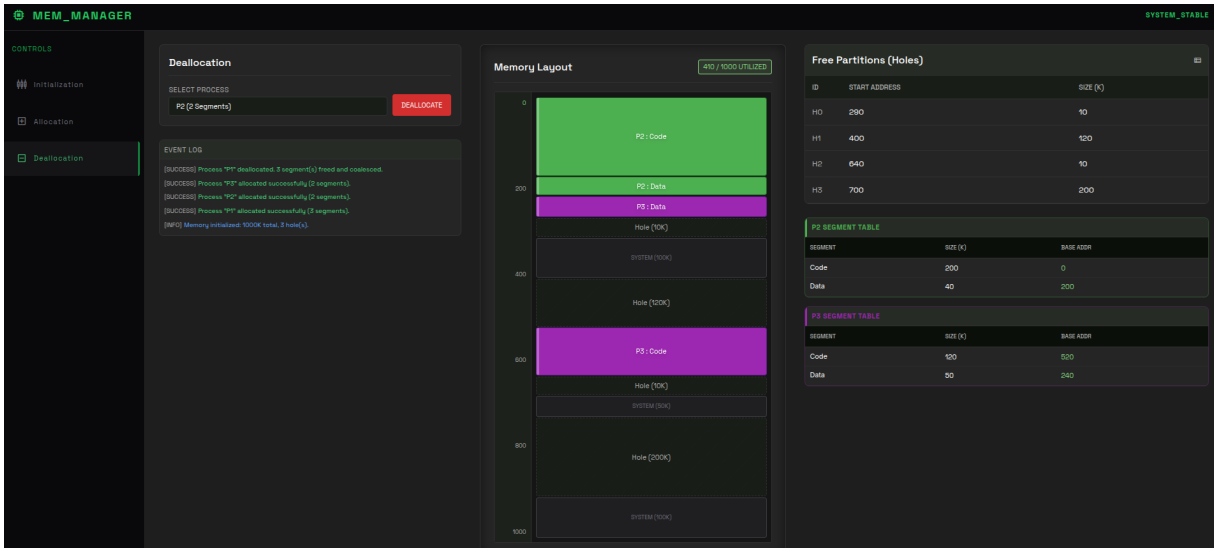


Figure 10: P1 deallocated with coalescing (Best-Fit scenario).

6.2.5 Step 5 – Allocate Process P4 (Code=230, Data=40) – FAILS

Code=230 cannot fit in any hole (largest is 200K). **P4 fails with Best-Fit** (it succeeded with First-Fit).

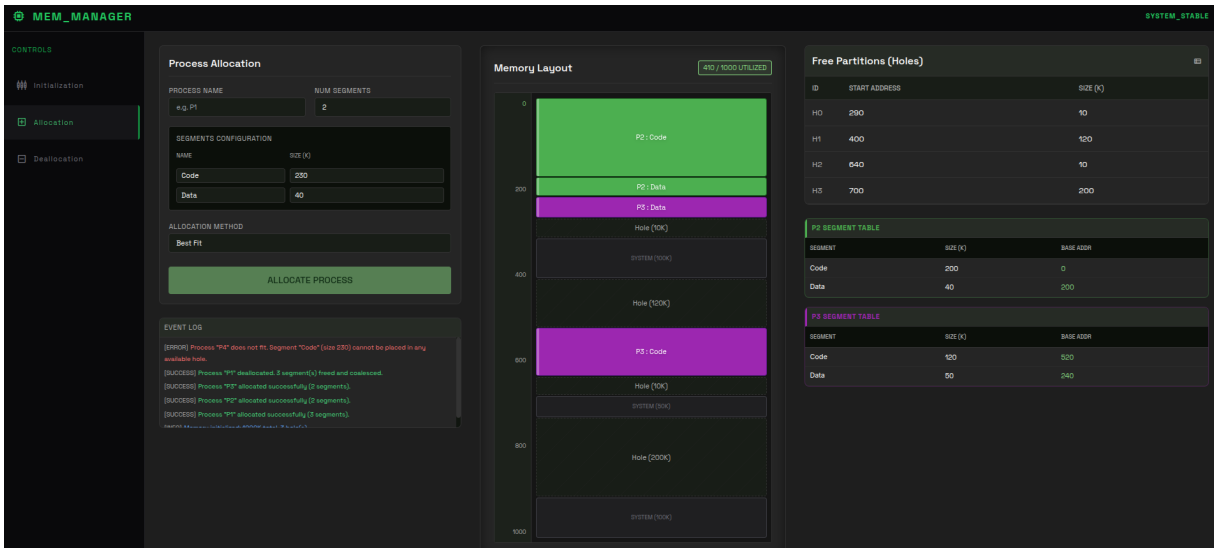


Figure 11: P4 allocation fails – demonstrates strategy trade-offs.

6.3 Test Results Summary

Operation	First-Fit	Best-Fit
P1 Allocate	✓	✓
P2 Allocate	✓	✓
P3 Allocate	× Fails	✓ Succeeds
P1 Deallocate	✓	✓
P4 Allocate	✓ Succeeds	× Fails

This demonstrates a key OS concept: **no single allocation strategy is universally better**. First-Fit is faster but can waste space; Best-Fit minimizes waste per allocation but can create tiny unusable fragments.

7 Technologies Used

Technology	Purpose
TypeScript	Strict type safety for both logic and UI
React 18	Component-based UI with hooks for state management
Vite	Lightning-fast build tool with HMR
Tailwind CSS	Utility-first dark-theme styling
Electron	Desktop wrapper (Chromium + Node.js)
electron-builder	Packaging into Windows .exe / Linux AppImage
GitHub Actions	CI/CD automated build pipeline

8 Source Code & Repository

The full source code, build instructions, and release artifacts are available at:

https://github.com/Mohamedkhaled687/OS_Memory_Segmentation_Simulator